

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. Create a New Java Project:

- Create a new Java project named **SingletonPatternExample**.

2. Define a Singleton Class:

- Create a class named **Logger** that has a private static instance of itself.
- Ensure the constructor of **Logger** is private.
- Provide a public static method to get the instance of the **Logger** class.

3. Implement the Singleton Pattern:

- Write code to ensure that the **Logger** class follows the Singleton design pattern.

4. Test the Singleton Implementation:

- Create a test class to verify that only one instance of **Logger** is created and used across the application.

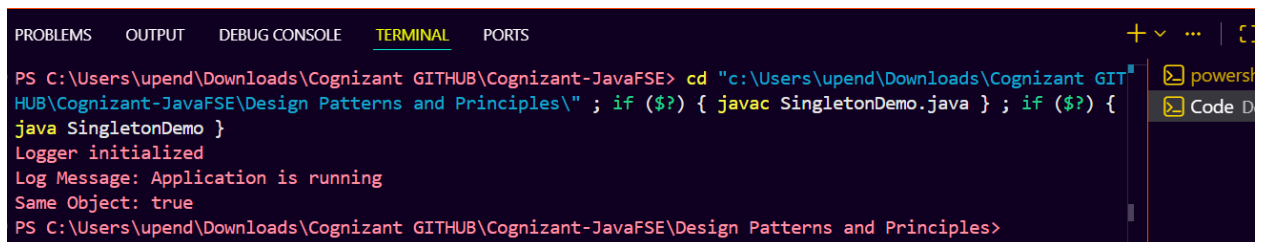
Code :

```
public class SingletonDemo {
    public static void main(String[] args) {
        AppLogger logger1 = AppLogger.getLogger();
        AppLogger logger2 = AppLogger.getLogger();
        logger1.writeLog("Application is running");
        System.out.println("Same Object: " + (logger1 == logger2));
    }
}

class AppLogger {
    private static AppLogger instance;
    private AppLogger() {
        System.out.println("Logger initialized");
    }
}
```

```
public static AppLogger getLogger() {  
    if (instance == null) {  
        instance = new AppLogger();  
    }  
    return instance;  
}  
  
public void writeLog(String message) {  
    System.out.println("Log Message: " + message);  
}  
}
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
PS C:\Users\upend\Downloads\Cognizant GITHUB\Cognizant-JavaFSE> cd "c:\Users\upend\Downloads\Cognizant GITH  
HUB\Cognizant-JavaFSE\Design Patterns and Principles\" ; if ($?) { javac SingletonDemo.java } ; if ($?) {  
java SingletonDemo }  
Logger initialized  
Log Message: Application is running  
Same Object: true  
PS C:\Users\upend\Downloads\Cognizant GITHUB\Cognizant-JavaFSE\Design Patterns and Principles>
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **FactoryMethodPatternExample**.
2. **Define Document Classes:**
 - Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.
3. **Create Concrete Document Classes:**
 - Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.
4. **Implement the Factory Method:**
 - Create an abstract class **DocumentFactory** with a method **createDocument()**.
 - Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.
5. **Test the Factory Method Implementation:**
 - Create a test class to demonstrate the creation of different document types using the factory method.

Code:

```
public class FactoryDemo {  
  
    public static void main(String[] args) {  
        FileFactory textFactory = new TextFactory();  
        FileFactory imageFactory = new ImageFactory();  
        FileFactory videoFactory = new VideoFactory();  
    }  
}
```

```
        FileType text = textFactory.createFile();
        FileType image = imageFactory.createFile();
        FileType video = videoFactory.createFile();
        text.openFile();
        image.openFile();
        video.openFile();
    }
}

interface FileType {
    void openFile();
}

class TextFile implements FileType {
    @Override
    public void openFile() {
        System.out.println("Opening Text File");
    }
}

class ImageFile implements FileType {
    @Override
    public void openFile() {
        System.out.println("Opening Image File");
    }
}

class VideoFile implements FileType {
    @Override
    public void openFile() {
        System.out.println("Opening Video File");
    }
}

abstract class FileFactory {
    public abstract FileType createFile();
}
```

```

}

class TextFactory extends FileFactory {
    @Override
    public FileType createFile() {
        return new TextFile();
    }
}

class ImageFactory extends FileFactory {
    @Override
    public FileType createFile() {
        return new ImageFile();
    }
}

class VideoFactory extends FileFactory {
    @Override
    public FileType createFile() {
        return new VideoFile();
    }
}

```

Output:

```

PS C:\Users\upend\Downloads\Cognizant GITHUB\Cognizant-JavaFSE\Design Patterns and Principles> cd "C:\Users\upend\Downloads\Cognizant GITHUB\Cognizant-JavaFSE\Design Patterns and Principles\" ; if ($?) { javac FactoryDemo.java } ; if ($?) { java FactoryDemo }
Opening Text File
Opening Image File
Opening Video File

```